

Kanban Deploy Runbook

How a commit on `main` becomes a running container on EC2 – the wiring, the credentials, and the one bug that cost an afternoon.

AWS ACCOUNT 239068269281	REGION us-east-1	ECR REPO kanban-app	INSTANCE i-052a30b10f906e2cf
HOST PORT 80 + 3000			

01 The pipeline

Five hops. The only interesting one is the third – proving to AWS that the build really is your repo, without storing a single credential anywhere.

LOCAL git push Merge to <code>main</code>	GITHUB CI runs format, lint, build	OIDC Assume role Short-lived token, no keys	ECR Image pushed Tagged <code>:latest</code> + SHA
EC2 Pull & run Manual, for now			

Pull requests run the first two stages only – they lint, format-check and build the image to prove it compiles, then stop. Nothing unmerged reaches the registry. That gate is a pair of `if: github.event_name == 'push'` conditions in the workflow.

02 Two roles, not one

The most common confusion in this setup: pushing and pulling are done by different identities, and neither one is you.

ROLE	WHO ASSUMES IT	CAN DO
<code>github-actions-ecr-push</code>	The GitHub Actions run, via OIDC	Push layers and images to the <code>kanban-app</code> repo only
<code>ec2-ecr-pull</code>	The EC2 instance, via instance profile	Read any ECR repo (<code>AmazonEC2ContainerRegistryReadOnly</code>)

The push role's inline policy is deliberately narrow.

`ecr:GetAuthorizationToken` must be `"Resource": "*" because logging in is an account-wide call; every other action` `BatchCheckLayerAvailability`, `InitiateLayerUpload`, `UploadLayerPart`, `CompleteLayerUpload`, `PutImage` is scoped to one repository ARN.

WHY OIDC

No access key ever exists. GitHub mints a signed token describing the workflow run; AWS verifies the signature and checks the claims against the role's trust policy. Nothing to leak, nothing to rotate.

03 The bug that cost an afternoon

Every value in the console was correct. The build failed anyway, eleven times.

The error was always the same line:

```
Error: Could not assume role with OIDC:
Not authorized to perform sts:AssumeRoleWithWebIdentity
```

AWS returns that identical message for *every* trust failure wrong audience, wrong subject, wrong role ARN, even a role that doesn't exist. It deliberately won't say which, to stop attackers enumerating roles. So the error carried no information, and checking the console repeatedly produced no new information either.

The breakthrough was to stop inspecting the AWS side and read what GitHub was actually *sending* – decoding the OIDC token's payload in a throwaway workflow step:

TRUST POLICY EXPECTED

repo:Nasir1504/kanban-board:*

TOKEN ACTUALLY CARRIED

repo:Nasir1504@87933341/kanban-board@1345866624:ref:refs/heads/main

The repo has GitHub's **immutable identifiers** enabled, so the subject claim embeds numeric account and repository IDs. The two strings could never match. Pasting the real subject into the trust policy fixed it on the next run.

KEEP IT THAT WAY

Numeric IDs are strictly better than names. They're never reissued, so a squatter can't claim a deleted repo name and inherit your role – and renaming the repo won't silently break the pipeline. Store the mapping somewhere: 87933341 = the account, 1345866624 = kanban-board.

NEXT TIME

The decoder step still lives in `ci.yml`, commented out. Uncomment it whenever a trust policy stops matching – after a rename, or when adding environment or tag-based triggers, both of which reshape the subject claim. It prints only `iss`, `aud`, `sub`, `repository` and `ref`; the token itself is never logged.

The transferable lesson: when one side refuses to say what it received, go and read what the other side sent.

04 What was configured

In order. Each step depends on the one before it, which is why they're numbered rather than bulleted.

1

OIDC identity provider

IAM Identity providers `token.actions.githubusercontent.com`, audience `sts.amazonaws.com`. One per account; shared by every repo.

2

Push role and its trust policy

Role `github-actions-ecr-push`, trusting the provider above. Actions allowed: `sts:AssumeRoleWithWebIdentity` and `sts:TagSession`, conditioned on the audience and the immutable subject.

3

ECR permissions

Inline policy on that role the two statements described in section 02.

4

GitHub repository variables

Variables, not secrets none of these are credentials. Referenced in the workflow as `${{ vars.NAME }}`.

NAME	VALUE
<code>AWS_REGION</code>	<code>us-east-1</code>
<code>AWS_ROLE_ARN</code>	<code>arn:aws:iam::239068269281:role/github-actions-ecr-push</code>
<code>ECR_REPOSITORY</code>	<code>kanban-app</code>

5

Workflow steps

Job-level `permissions: id-token: write` omit it and you get a misleading "credentials could not be loaded". Then `configure-aws-credentials`, ECR login, build and push.

6

Instance pull role

Role `ec2-ecr-pull` with `AmazonEC2ContainerRegistryReadOnly`, attached via EC2 Actions Security Modify IAM role. Creating the role does not attach it that's a separate action.

7

Instance software

Ubuntu instance. Docker from apt; AWS CLI v2 from the official zip â€” `apt install awscli` gives the outdated v1.

```
sudo apt update && sudo apt install -y docker.io unzip curl
sudo systemctl enable --now docker
sudo usermod -aG docker ubuntu

curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip -q awscliv2.zip && sudo ./aws/install
```

Log out and back in after `usermod`, or the group change won't apply.

8

Environment file

`~/ .env` on the instance, holding `MONGODB_URI`. It is never baked into the image â€” `--env-file` reads it on the host and injects the values as environment variables, so there is no `.env` inside the container. Check with `docker exec kanban printenv MONGODB_URI`.

05 Day-to-day commands

Run on the instance over SSH.

Deploy a new version

After CI has pushed a fresh image. Containers are replaced, never edited in place.

```
aws ecr get-login-password --region us-east-1 \
  | docker login --username AWS --password-stdin 239068269281.dkr.ecr.us-east-1.amazonaws.com

docker pull 239068269281.dkr.ecr.us-east-1.amazonaws.com/kanban-app:latest
docker stop kanban && docker rm kanban

docker run -d --name kanban --restart unless-stopped \
  -p 80:3000 --env-file ~/.env -e HOSTNAME=0.0.0.0 \
  239068269281.dkr.ecr.us-east-1.amazonaws.com/kanban-app:latest
```

Inspect

```
docker ps          # want: Up, 0.0.0.0:80→3000/tcp
docker logs kanban  # startup errors, DB connection failures
curl -I http://localhost # want: HTTP/1.1 200 OK
docker exec -it kanban sh # /app holds server.js, .next, public
```

Stop and start

```
docker stop kanban # stays down; unless-stopped respects a manual stop
docker start kanban
```

TWO FLAGS WORTH REMEMBERING

`-e HOSTNAME=0.0.0.0` â€” Next.js standalone can bind to localhost inside the container, which port mapping cannot reach. Without it the container runs and the port still gives nothing.

`--restart unless-stopped` â€” survives reboots, but honours a deliberate `docker stop`.

THE IMAGE IS NOT YOUR SOURCE

`/app` contains the compiled Next.js standalone output â€” around five entries, which is correct. Your `.tsx` files aren't in there; they existed only in the builder stage. Edit locally, push, let CI rebuild.

06 Still open

The app serves a 200 on port 80. These are the gaps between that and a deployment you'd leave running.

ITEM	STATUS	WHAT'S NEEDED
------	--------	---------------

Database	BROKEN	<code>MONGODB_URI</code> still points at <code>localhost:27017</code> . Nothing is listening there, and inside a container <code>localhost</code> means the container itself. Use Atlas (allowlist the instance IP) or run Mongo as a second container on a shared Docker network.
----------	---------------	--

ITEM	STATUS	WHAT'S NEEDED
Stable address	FRAGILE	No Elastic IP. Stop and start the instance and the public IP changes “” which breaks any DNS record pointing at it.
HTTPS	ABSENT	Certificates are issued to domains, never to bare IPs, so a domain comes first. Then Caddy on the instance (free, auto-renewing) or an ALB with an ACM certificate (~\$16/mo).
Deploy to EC2	MANUAL	CI stops at ECR. Pulling and restarting is done by hand over SSH. Automate later with SSM Run Command, or a Watchtower-style poller.
CI “ ECR	WORKING	Green on <code>main</code> . Two tags per build: <code>latest</code> and the commit SHA.

Set up 2 September 2026 “ `Nasir1504/kanban-board` “ workflow at `.github/workflows/ci.yml`